# Dokumentuak: MongoDB

Mikel Egaña Aranguren

mikel-egana-aranguren.github.io

mikel.egana@ehu.eus

Universidad del País Vasco
Euskal Herriko Unibertsitatea

BILBOKO INGENIARITZA ESKOLA
ESCUELA DE INGENIERÍA DE BILBAO

# Dokumentuak: MongoDB

https://github.com/mikel-egana-aranguren/DBA

cc BY

# MongoDB

**Dokumentuetan** oinarritutako NoSQL datu-basea: datuak JSON dokumentu gisa gordetzen dira

Ez du eskema finkorik (schemaless)

# Tresnak

- **mongod** — datu basearen zerbitzaria
- **mongosh** — shell interaktiboa komandoak exekutatzeko
- **MongoDB Compass** — interfaze grafikoa kolekzioak aztertzeko
- **MongoDB Atlas** — kudeaturiko cloud zerbitzua

# Datu eredua

| Kontzeptua | MongoDB |
|---|---|
| Datu basea | Datu basea |
| Antolakuntza unitatea | Kolekzioa |
| Datu unitatea | Dokumentua |
| Atributua | Eremua (field) |
| Identifikatzailea | _id |
| Erlazioak | Dokumentuen arteko erlazioak: dokumentu txertatuak / erreferentziak |

# JSON: JavaScript Object Notation

Datu-trukerako formatu arina, gizakientzat eta makinentzat irakurgarria

JSON (ECMA 404)

Gako-balio bikoteetan oinarritua

Datu motak: string-ak, zenbakiak, boolearrak, array-ak, objektuak, null

Objektuak beste objektuak eta objektuen array-ak eduki ditzakete: datuak jerarkikoki zuzenean modelatzea

# JSON dokumentuak

```
{
  "nombre": "Javier Cansado",
  "direccion": {
    "calle": "Gran Vía 1",
    "ciudad": "Bilbao",
    "cp": "48001"
  },
  "telefonos": [
    {"tipo": "movil", "numero": "666111222"},
    {"tipo": "casa", "numero": "944123456"}
  ],
  "viviendas": ["Deusto", "Abando", "Begoña"]
}
```

# BSON: Binary JSON

MongoDB-k dokumentuak barnean BSON formatuan gordetzen ditu

JSON-aren adierazpen binarioa, biltegirako eraginkorragoa

# BSON: Binary JSON

JSON-ek ez dituen datatype osagarriak onartzen ditu:

- `ObjectId` – 12 byteko identifikatzaile bakarra
- `Date` – milisegundoko doitasuna duten datak
- `NumberInt`, `NumberLong`, `NumberDecimal`
- `BinData` – datu binarioak
- `Regex` – adierazpen erregularrak

# ObjectId

Dokumentu bakoitzak `_id` eremua du gako nagusi bezala

Ez badago JSON dokumentuan, MongoDB-k `ObjectId` bat automatikoki sortzen du

Datu Base osoan bakarra da, sorreraren ordenen arabera ordenatua

Automatikoki sortzen da eta banatutako sistemetan ondo funtzionatzen du

# Konektatu eta datu-base bat sortu

```
// Konektatu zerbitzarira (Terminal edo GUI)
$ mongosh
$ mongodb-compass

// MongoDB-n datu basea erabiltzea nahikoa da (txertatzean sortzen da)
test> use ehu

// Uneko datu basearen kolekzioak ikusi
ehu> show collections
```

MongoDB-k datu-baseak eta kolekzioak modu inplizituan sortzen ditu

lehenengo dokumentua txertatzean

# CRUD: Oinarrizko eragiketak

| Eragiketa | MongoDB metodoak |
|---|---|
| Sortu | insertOne() / insertMany() |
| Irakurri | find() / findOne() |
| Eguneratu | updateOne() / updateMany() |
| Ezabatu | deleteOne() / deleteMany() |

# insertOne()

```
db.alumni.insertOne({
  nombre: "Javier Cansado",
  email: "javi@ehu.eus",
  edad: 21
})
```

Ez da beharrezkoa aurretik kolekzioa definitzea

Ez da beharrezkoa dokumentu guztiek eremu berberak izatea

MongoDB-k ID bat automatikoki txertatzen du

db.alumni.findOne({ _id: ObjectId("XXX") }) / db.alumni.find()

# insertMany()

```
db.alumni.insertMany([
  {
    nombre: "Jon Aguirre",
    email: "jon@ehu.eus",
    edad: 22,
    notas: [
      { asignatura: "ABD", nota: 7.5 }
    ]
  },
  {
    nombre: "Miren Etxebarria",
    email: "miren@ehu.eus",
    edad: 20
  },
  {
    nombre: "Unai López",
    email: "unai@ehu.eus",
    edad: 23,
    becado: true
  }
])
```

# Schemaless: malgukortasuna

Dokumentu bakoitzak eremu desberdinak eduki ditzake

Guztiak kolekzio berean arazorik gabe bizi daitezke

# find(): dokumentuak kontsultatu

```
db.alumni.find()

db.alumni.find({ edad: 20 })

db.alumni.find({ nombre: "Jon Aguirre" })

// findOne() lehenengo emaitza soilik itzultzen du
```

# Alderapen operadoreak

```
db.alumni.find({ edad: { $gt: 21 } })

db.alumni.find({ edad: { $gte: 20, $lte: 23 } })

db.alumni.find({ edad: { $ne: 21 } })

db.alumni.find({ edad: { $in: [20, 21, 22] } })
```

`$gt`, `$gte`, `$lt`, `$lte`, `$eq`, `$ne`, `$in`, `$nin`

# Logika operadoreak

```
db.alumni.find({
  $and: [
    { edad: { $gt: 20 } },
    { nombre: "Javier Cansado" }
  ]
})

db.alumni.find({
  $or: [
    { edad: { $lt: 22 } },
    { becado: true }
  ]
})
```

`$and`, `$or`, `$not`, `$nor`

# AND inplizitua

Eremu desberdinak daudenean, MongoDB-k AND aplikatzen du:

```
db.alumni.find({ edad: 21, nombre: "Javier Cansado" })


db.alumni.find({
  $and: [
    { edad: 21 },
    { nombre: "Javier Cansado" }
  ]
})
```

# Proiekzioa (eremuak aukeratu)

```
db.alumni.find(
  {}, // iragazkia: guztiak
  { nombre: 1, email: 1 } // proiekzioa: eremu hauek bakarrik
)

// Eremuak baztertu (guztia erakutsi email kenduta):
db.alumni.find(
  {},
  { email: 0 }
)

// _id eremua lehenespenez sartzen da
// Baztertzeko:
db.alumni.find({}, { nombre: 1, _id: 0 })
```

1 = sartu, 0 = baztertu. Ezin dira nahastu (_id izan ezik)

# Ordenatu, mugatu

```
db.alumni.find().sort({ edad: 1 })

db.alumni.find().sort({ edad: -1 })

db.alumni.find().limit(3)

db.alumni.find().sort({ edad: -1 }).limit(2)
```

# Dokumentuak zenbatu

```
db.alumni.countDocuments()

db.alumni.countDocuments({ edad: { $gt: 21 } })

// Estimazio kalkuluarekin zenbatu (kolekzio handientzat):
db.alumni.estimatedDocumentCount()
```

# updateOne()

```
db.alumni.updateOne(
  { nombre: "Javier Cansado" }, // iragazkia
  { $set: { edad: 22 } } // eguneraketa (UPSERT)
)

// $set -ek adierazitako eremuak bakarrik aldatzen ditu
// Eremua ez badago, sortzen du:
db.alumni.updateOne(
  { nombre: "Javier Cansado" },
  { $set: { telefono: "666111222" } }
)
```

# Eguneraketa operadoreak

```
// Zenbaki balioa gehitu
db.alumni.updateOne(
  { nombre: "Jon Aguirre" },
  { $inc: { edad: 1 } }
)

// Dokumentutik eremua ezabatu
db.alumni.updateOne(
  { nombre: "Unai López" },
  { $unset: { becado: "" } }
)

// Eremua berrizzentzatu
db.alumni.updateOne(
  { nombre: "Javier Cansado" },
  { $rename: { "telefono": "tlfno" } }
)
```

`$set`, `$unset`, `$inc`, `$rename`, `$min`, `$max`, `$mul`

# updateMany()

```
db.alumni.updateMany(
  { edad: { $gte: 20 } },
  { $set: { activo: true } }
)

// Emaitza:
// { acknowledged: true,
// matchedCount: 4,
// modifiedCount: 4 }
```

`updateOne`-k bat datorren lehen dokumentua soilik aldatzen du

`updateMany` bat datozen dokumentu guztiak aldatzen ditu

# Array-en arteko eragiketak

```
// Elementu bat array-an gehitu
db.alumni.updateOne(
  { nombre: "Jon Aguirre" },
  { $push: { notas: { asignatura: "SI", nota: 8.0 } } }
)

// Hainbat elementu array-an gehitu
db.alumni.updateOne(
  { nombre: "Jon Aguirre" },
  { $push: { notas: { $each: [
    { asignatura: "ED", nota: 6.5 },
    { asignatura: "SO", nota: 7.0 }
  ]}}}
)

// Elementu bat array-an ezabatu
db.alumni.updateOne(
  { nombre: "Jon Aguirre" },
  { $pull: { notas: { asignatura: "ED" } } }
)
```

`$push`, `$pull`, `$addToSet`, `$pop`, `$each`

# deleteOne() eta deleteMany()

```
db.alumni.deleteOne({ nombre: "Unai López" })

db.alumni.deleteMany({ edad: { $lt: 20 } })

// Kolekzioko dokumentu guztiak ezabatu
db.alumni.deleteMany({})

// Kolekzio osoa ezabatu
db.alumni.drop()

// Uneko datu basea ezabatu
db.dropDatabase()
```

# replaceOne()

```
// Dokumentu OSO bat ordezten du (ez banakako eremuak)
db.alumni.replaceOne(
  { nombre: "Javier Cansado" },
  {
    nombre: "Javier Cansado Fernández",
    email: "javier.cansado.fernandez@ehu.eus",
    edad: 22,
    grado: "Ingeniería Informática"
  }
)
```

**Kontuz:** `replaceOne()`-k dokumentu osoa ordezkatzen du

`updateOne()` + `$set` erabiliz, eremu espezifikoak bakarrik aldatzen dira

# Dokumentu bateratuen kontsultak

```
Datuak: mongodb-compass; gehitu datuak; txertatu dokumentua (nested.json)

// Eremu bateratua bilatu (dot notation):
db.alumni.find({ "direccion.ciudad": "Bilbao" }) // mongosh
{ "direccion.ciudad": "Bilbao" } // mongodb-compass
```

Dot notation: `"eremua.azpi-eremua"` eremu kapsulatuetara sartzeko

# Array-etan kontsultak

```
Datuak: nested-arrays.json (mongodb-compass: inportatu)

// "euskera" array-an edukitzea dokumentuak bilatu:
db.alumni.find({ idiomas: "euskera" })
// Emaitza: Jon

// Array-an balio zehatzak duten dokumentuak bilatu:
db.alumni.find({ idiomas: ["castellano", "francés"] })
// Emaitza: Mikel (ordenak garrantzia du)

// Array-aren tamainaren bidez bilatu:
db.alumni.find({ idiomas: { $size: 3 } })
// Emaitza: Jon (3 hizkuntza ditu)
```

# Objektuen array-etan kontsultak

```
{
  nombre: "Jon",
  notas: [
    { asignatura: "ABD", nota: 7.5 },
    { asignatura: "SI", nota: 8.0 }
    { asignatura: "SO", nota: 7.0 }
  ]
}

// ehu bilatu nota > 7 duten ikasgaia batean:
db.alumni.find({ "notas.nota": { $gt: 7 } })

// ehu bilatu nota > 7 duten ABD zehatz-mehatz:
db.alumni.find({
  notas: {
    $elemMatch: {
      asignatura: "ABD",
      nota: { $gt: 7 }
    }
  }
})
```

`$elemMatch` : elementu berak baldintza guztiak bete behar ditu

# $exists eta $type

```
// "becado" eremua DUTEN dokumentuak bilatu:
db.alumni.find({ becado: { $exists: true } })

// "telefono" eremua ez DUTEN dokumentuak bilatu:
db.alumni.find({ telefono: { $exists: false } })

// "edad" zenbaki motatakoa den dokumentuak bilatu:
db.alumni.find({ edad: { $type: "number" } })

// "nombre" string bat den dokumentuak bilatu:
db.alumni.find({ nombre: { $type: "string" } })
```

Oso baliagarriak dira schemaless kolekzioetan, dokumentu guztiek ez baitituzte eremu berak

# Adierazpen erregularrak

```
db.alumni.find({ nombre: /^Mi*/ })

db.alumni.find({ email: /ehu\.eus$/ })

// Maiuskula minuskula ez den bilaketa:
db.alumni.find({ nombre: { $regex: "Cansado", $options: "i" } })
```

PCRE (Perl Compatible Regular Expressions) adierazpen erregularrak

# Aggregation Pipeline

Etapetan (stages) datuak prozesatzeko sistema

Dokumentuak transformazioen kate bat zeharkatzen dute

Datuak iragaztea, eraldatzea, taldekatzea eta aberastea pausoz pauso ahalbidetzen du

```
db.kolekzioa.aggregate([ { $etapa1: { ... } }, { $etapa2: { ... } }, { $etapa3: { ... } } ])
```

Etapa baten irteera hurrengoarena da

# Pipeline-ren etapa nagusiak

| Etapa | Deskribapena |
|---|---|
| `$match` | Dokumentuak iragazi |
| `$group` | Taldekatu eta kalkulatu |
| `$project` | Eremuak aukeratu/aldatu |
| `$sort` | Emaitzak ordenatu |
| `$limit` | Emaitzak mugatu |
| `$unwind` | Array-ak deskonposatu |
| `$lookup` | Kolekzioak batzea |

# $match: iragaztea

```
db.alumni.aggregate([
  { $match: { edad: { $gt: 20 } } }
])

// Iragazkiaren sintaxia find()-n berdin da
// $match ahalik eta lehenengo joan beharko litzateke murrizteko
// hurrengo etapetara pasatzen diren dokumentuen kopurua
```

# $group: taldekatu eta kalkulatu

```
"pedidos" kolekzio berria, pedidos.json kargatu

db.pedidos.aggregate([
  {
    $group: {
      _id: "$cliente",
      total: { $sum: "$precio" },
      num_pedidos: { $sum: 1 }
    }
  }
])
```

# $group funtzioak

| MongoDB | Deskribapena |
|---|---|
| `$sum` | Balioak batu edo zenbatu |
| `$avg` | Batezbestekoa |
| `$min` | Gutxieneko balioa |
| `$max` | Gehieneko balioa |
| `$first` | Taldearen lehenengo balioa |
| `$push` | Balio guztien array-a |

# Pipeline osoa: adibidea

```
db.pedidos.aggregate([
  { $match: { precio: { $gt: 30 } } },            // WHERE
  { $group: {                              // GROUP BY
      _id: "$cliente",
      total: { $sum: "$precio" }
  }},
  { $match: { total: { $gt: 200 } } },        // HAVING
  { $sort: { total: -1 } }                    // ORDER BY
])
```

Pipeline-ak kontsulta konplexuak pausoz pauso eraikitzea ahalbidetzen du

# $project: eremuak aldatu

```
db.alumni.aggregate([
  {
    $project: {
      nombre: 1,
      email_mayus: { $toUpper: "$email" },
      anio_nacimiento: {
        $subtract: [2026, "$edad"]
      },
      _id: 0
    }
  }
])
```

# $unwind: array-ak deskonposatu

```
// Datua: { nombre: "Jon", notas: [{asig: "ABD", nota: 7.5},
//                                 {asig: "SI", nota: 8.0}] }
// $unwind array-aren eremu bakoitzerako dokumentua sortzen du:

db.alumni.aggregate([
  { $unwind: "$notas" }
])
// Emaitza:
// { nombre: "Jon", notas: { asig: "ABD", nota: 7.5 } }
// { nombre: "Jon", notas: { asig: "SI", nota: 8.0 } }

// Geroago ikasgaian $group egiteko erabilgarria:
db.alumni.aggregate([
  { $unwind: "$notas" },
  { $group: {
      _id: "$notas.asignatura",
      media: { $avg: "$notas.nota" }
  }}
])
```

# $lookup: kolekzioak batzea

```
db.pedidos.aggregate([
   {
    $lookup: {
         from: "alumni",
         localField: "cliente",
         foreignField: "nombre",
         as: "alumniMatch"
     }
   }
])
```

# MongoDB-n eskema diseinua

MongoDB schemaless bada ere, eskema diseinua oinarrizkoa da

Gako-galdera: **txertatu** ala **erreferentziatu**?

- **Txertatu**: erlazionatutako datuak dokumentuaren barruan sartu
- **Erreferentziatu**: erlazionatutako dokumentuaren _id gorde

Aukera atzipen-patroietan, tamainan eta eguneratze-maiztasunean oinarritzen da

# Txertatzea (desnormalizatzea)

```
// Informazio guztia dokumentu bakarrean
{
  nombre: "Javier Cansado",
  direccion: {
    calle: "Gran Vía 1",
    ciudad: "Bilbao"
  },
  pedidos: [
    { producto: "Monitor", precio: 250, fecha: "2026-01-15" },
    { producto: "Ratón", precio: 25, fecha: "2026-02-20" }
  ]
}
```

**Abantailak:** irakurketa oso azkarra (kontsulta bakarra)

**Mugak:** datuen bikoizketa eta dokumentu handiak

# Erreferentziatzea (normalizatzea)

```
// "clientes" kolekzioa
{
  _id: ObjectId("aaa..."),
  nombre: "Javier Cansado",
  direccion: { calle: "Gran Vía 1", ciudad: "Bilbao" }
}

// "pedidos" kolekzioa
{
  cliente_id: ObjectId("aaa..."),
  producto: "Monitor",
  precio: 250,
  fecha: "2026-01-15"
}
```

**Abantailak:** bikoizketarik gabe, dokumentu txikiagoak

**Mugak:** $lookup behar du (motelagoa) eta kontsulta gehiago

# Noiz txertatu? Noiz erreferentziatu?

| Irizpidea | Txertatu | Erreferentzia |
|---|---|---|
| Erlazioa | 1:1, 1: gutxi | 1: asko, N:M |
| Datuak elkarrekin irakurtzen dira | Bai | Ez beti |
| Datuak asko aldatzen dira | Ez | Bai |
| Subdatu neurria | Txikia/mugatua | Handia/mugagabea |
| Lehentasuna | Irakurketa abiadura | Koherentzia |

# MongoDB-n indizeak

Indizerik gabe, MongoDB-k dokumentu guztiak aztertu behar ditu (collection scan)

Indize batek dokumentuak aurkitzea ahalbidetzen du, kolekzio osoa eskaneatu gabe

MongoDB-k automatikoki _id gainean indize bat sortzen du

Edozein eremuren gainean edo eremuen konbinazioetan indizeak sor daitezke

# Indizeak sortu eta kudeatu

```
// Kargatu ehu.alumni.json ehu.alumni1000 kolekzioan

// Existitzen diren indizeak ikusi:
db.alumni1000.getIndexes()

// Indize sinplea sortu:
db.alumni1000.createIndex({ email: 1 }) // 1 = goraka; -1 = behera

// Indize konposatua (eremu anitzetan):
db.alumni1000.createIndex({ nombre: 1, edad: -1 })

// Indize bakarra (ez du bikoiztua baimentzen):
db.alumni1000.createIndex({ email: 1 }, { unique: true })

// Indizea ezabatu:
db.alumni1000.dropIndex({ email: 1 })
```

# explain(): kontsultak aztertu

```
db.alumni1000.find({ email: "alumno0680@ikasle.ehu.eus" }).explain("executionStats")
// COLLSCAN: indizerik gabe, kolekzio osoa eskaneatzen du
// IXSCAN: indizearekin, zuzenean bilatzen du
```

Kontsultetan maiz erabiltzen diren eremuetan indizeak sortu

Indizeek irakurketak bizkortzen dituzte, baina idazketak moteldu

# Eskema egiaztapena

MongoDB schemaless bada ere, balidazio aukerakoa defini daiteke

JSON Schema erabil daiteke dokumentuetan arauak definitzeko

```
db.createCollection("ehu-schema", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["nombre", "email", "edad"],
      properties: {
        nombre: { bsonType: "string",
                  description: "string bat izan behar du" },
        email:  { bsonType: "string",
                  pattern: "^.+@.+$" },
        edad:   { bsonType: "int",
                  minimum: 16, maximum: 99 }
      }
    }
  }
})
```

# Egiaztapena: mailak

"strict" maila (lehenespenez): insert eta update eragiketetan balioztatzen du

"moderate" maila: dokumentu berriak eta lehendik baliozkoak diren dokumentuak balioztatzen ditu

Eskema malgukortasuna eta datuen integritatea orekatzea ahalbidetzen du

collMod_moderate, collMod_strict, productos_ejemplo, invalid_productos

# MongoDB erabilera egokia

- Dokumentuak kontsulta-patroi ohikoenen arabera diseinatu
- Txertatzearen eta erreferentziatzearen artean aukeratu atzipenaren eta tamainaren irizpideen arabera
- Iragazkietan eta ordenazioan erabilitako eremuetan indizeak sortu
- Integritatea kritikoa denean eskema egiaztapena aplikatu
- explain() eta errendimendu-metrikak erabiliz kontsultak monitorizatu

# MongoDB / Triple Stores

RDF (JSON-LD) artxibo bat MongoDB-n kargatu al daiteke?

GraphDB / Ontop?